

Festival

Nayra está en un festival jugando a un juego en el que el premio mayor es un viaje a Laguna Colorada. El juego consiste en usar tokens para comprar cupones. Comprar un cupón podría aumentar la cantidad de tokens. La meta es obtener la mayor cantidad de cupones como sea posible.

Ella empieza el juego con A tokens, los cuales puede usar para comprar cupones. Hay N cupones enumerados del 0 al $N - 1$. Nayra debe pagar $P[i]$ tokens para comprar el i -ésimo cupón (y debe tener al menos $P[i]$ tokens antes de la compra). Ella puede comprar cada cupón como máximo una vez.

Además, el i -ésimo cupón ($0 \leq i < N$) tiene asignado un **tipo**, denotado por $T[i]$, el cual es un entero **entre 1 y 4, incluyendo los extremos**.

Luego de que Nayra compra el cupón i , la cantidad restante de tokens que tiene es multiplicada por $T[i]$. Formalmente, si ella tiene X tokens en un punto del juego, y compra el i -ésimo cupón (lo cual requiere $X \geq P[i]$), entonces ella tendrá $(X - P[i]) \cdot T[i]$ tokens luego de la compra.

Tu tarea es determinar cuáles cupones debe comprar Nayra y en qué orden para maximizar la cantidad total de **cupones** que tendrá al final del juego. Si hay más de una secuencia de compras que logre esto, puedes reportar cualquiera de ellas.

Detalles de la Implementación

Debes implementar la siguiente función.

```
std::vector<int> max_coupons(int A, std::vector<int> P,  
                             std::vector<int> T)
```

- A : la cantidad inicial de tokens que tiene Nayra.
- P : un arreglo de longitud N especificando los precios de los cupones.
- T : un arreglo de longitud N especificando los tipos de los cupones.
- Esta función es llamada exactamente una vez para cada caso de prueba.

Esta función debe devolver un arreglo R que especifique las compras de Nayra de la siguiente manera:

- La longitud de R debe ser la máxima cantidad de cupones que ella puede comprar.
- Los elementos del arreglo son los índices de los cupones que Nayra debe comprar en orden cronológico. Esto es, que ella compra el cupón $R[0]$ primero, el $R[1]$ segundo y así.
- Todos los elementos de R deben ser únicos.

Si no se compran cupones, R debe ser un arreglo vacío.

Restricciones

- $1 \leq N \leq 200\,000$
- $1 \leq A \leq 10^9$
- $1 \leq P[i] \leq 10^9$ para cada i tal que $0 \leq i < N$.
- $1 \leq T[i] \leq 4$ para cada i tal que $0 \leq i < N$.

Subtareas

Subtarea	Puntaje	Restricciones adicionales
1	5	$T[i] = 1$ para cada i tal que $0 \leq i < N$.
2	7	$N \leq 3000$; $T[i] \leq 2$ para cada i tal que $0 \leq i < N$.
3	12	$T[i] \leq 2$ para cada i tal que $0 \leq i < N$.
4	15	$N \leq 70$
5	27	Nayra puede comprar todos los N cupones (en algún orden).
6	16	$(A - P[i]) \cdot T[i] < A$ para cada i tal que $0 \leq i < N$.
7	18	Sin restricciones adicionales

Ejemplos

Ejemplo 1

Considera la siguiente llamada.

```
max_coupons(13, [4, 500, 8, 14], [1, 3, 3, 4])
```

Inicialmente, Nayra tiene $A = 13$ tokens. Ella puede comprar 3 cupones en el orden mostrado a continuación:

Cupón comprado	Precio del cupón	Tipo del cupón	Cantidad de tokens después de la compra
2	8	3	$(13 - 8) \cdot 3 = 15$
3	14	4	$(15 - 14) \cdot 4 = 4$
0	4	1	$(4 - 4) \cdot 1 = 0$

En este ejemplo, Nayra no tiene posibilidad de comprar más de 3 cupones y la secuencia de compras descrita anteriormente es la única forma en la que puede comprar los 3. Por lo tanto, la función debe devolver $[2, 3, 0]$.

Ejemplo 2

Considera la siguiente llamada.

```
max_coupons(9, [6, 5], [2, 3])
```

En este ejemplo, Nayra puede comprar los dos cupones en cualquier orden. Por lo tanto, la función debe devolver $[0, 1]$ o $[1, 0]$.

Ejemplo 3

Considera la siguiente llamada.

```
max_coupons(1, [2, 5, 7], [4, 3, 1])
```

En este ejemplo, Nayra tiene un token el cual no es suficiente para comprar algún cupón. Por lo tanto, la función debe devolver $[]$ (un arreglo vacío).

Grader de Ejemplo

Formato de entrada:

```
N A
P[0] T[0]
P[1] T[1]
...
P[N-1] T[N-1]
```

Formato de salida:

```
S
R[0]  R[1]  ...  R[S-1]
```

Aquí, S es la longitud del arreglo R devuelto por `max_coupons`.